

IBM BOB 2.0 HACKATHON

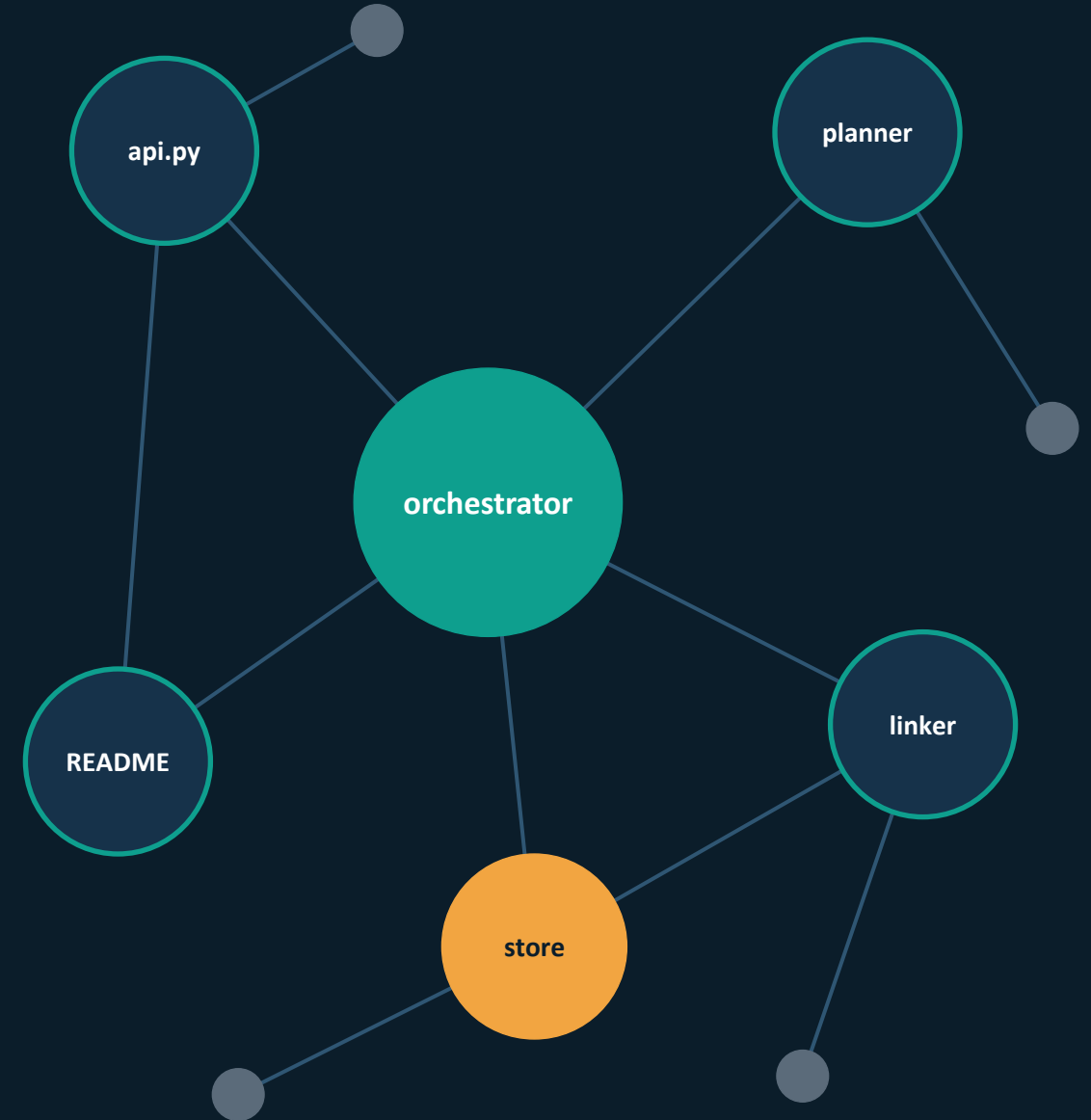
Cartograph

Map any codebase into a knowledge graph — then ask it questions and get answers with citations.

Read once. Answer from the graph. Refuse to bluff.

Built with IBM Bob 2.0 · watsonx.ai-ready · Bob Shell provider

Team Poornasri2003 · github.com/Poornasri2003/code-analyser



The same codebase, re-learned every time



Onboarding

A new joiner spends their first days finding the entry point, the call paths and the one module that matters.



Inheriting a service

The original author moved on. The only map of the system left with them.



Reviewing a PR

The reviewer has never read this module, so they approve the diff without seeing what it touches.

“

The mental map one developer builds is never written down — so the next person rebuilds it from scratch.

Tools today: grep, the file tree, and whoever is free to answer questions.

Pasting the repo into a chat assistant

Chat over raw files

✗ Re-reads raw files for every question

✗ Context window caps what fits

✗ Fluent answer, no source

✗ The work is invisible

Cartograph

✓ Reads the repo once; answers come from the graph

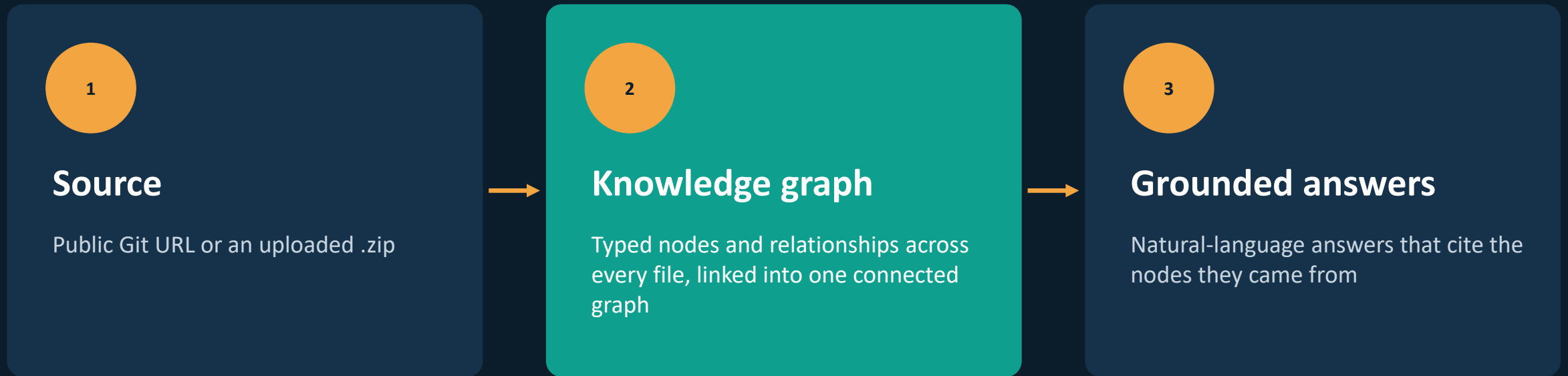
✓ Retrieves only the relevant subgraph, so large repos still work

✓ Every claim cites a graph node, or is flagged ungrounded

✓ Live trace, token spend and node/edge counts for every run

Read once. Answer forever.

Cartograph turns a repository into a queryable knowledge graph, then answers from the graph instead of re-reading the source.

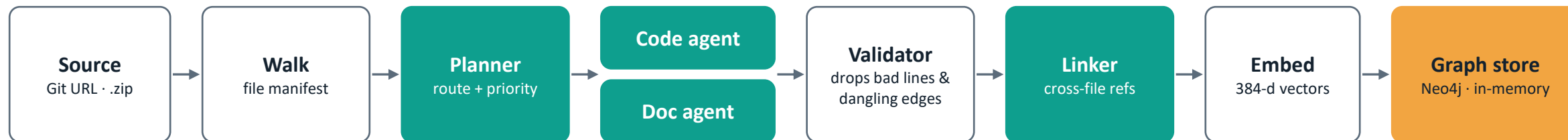


Two pipelines, one graph

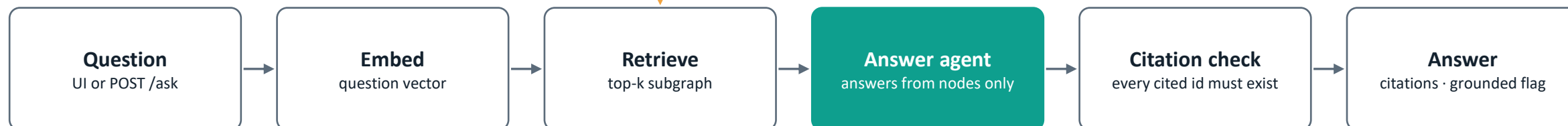
 LLM agent

 Deterministic Python

ANALYSE · once per repository

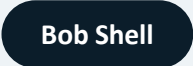


ASK · every question



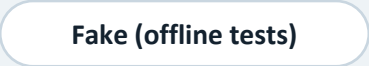
LLMClient protocol — one interface, any provider

 watsonx.ai · Granite (default)

 Bob Shell

 Groq

 OpenAI-compatible

 Fake (offline tests)

Interfaces

```
FastAPI + web UI · CLI
/analyse /analyse-upload /jobs /trace
/graph /ask
```

Orchestrator owns order, retries, validation, ids and storage. Agents only return JSON.

The LLM reasons. Python decides.



Planner

Sees the whole file tree. Routes each file to code, doc or skip (with a reason) and ranks it 1–3.



Code agent

Turns a source file into typed nodes (modules, classes, functions) with line ranges.



Doc agent

Turns prose, configs and schemas into concepts and the code they describe.



Linker

Resolves references across files, merges duplicates, drops what cannot be resolved.



Answer agent

Answers only from the retrieved subgraph and cites node ids.

A malformed reply is retried once, then logged as a failure in the run report. The run keeps going.

Deterministic tools

`resolve_source` · `walk_files`

clone or unzip, list files

`read_file`

path-traversal checked

`search` · `find_references`

ripgrep with Python fallback

`validate_graph`

rejects impossible line ranges

`assign_ids`

stable SHA-1 ids per run

`embed`

384-d hashing or MiniLM

A graph that refuses to bluff

1

Fail-closed grounding

If the model cites a node that was not in the retrieved context, the answer comes back grounded=false instead of being shown as fact.

2

Validated before stored

Hallucinated line ranges and edges to nodes that do not exist are dropped, and counted in the run report.

3

Priority-first budget

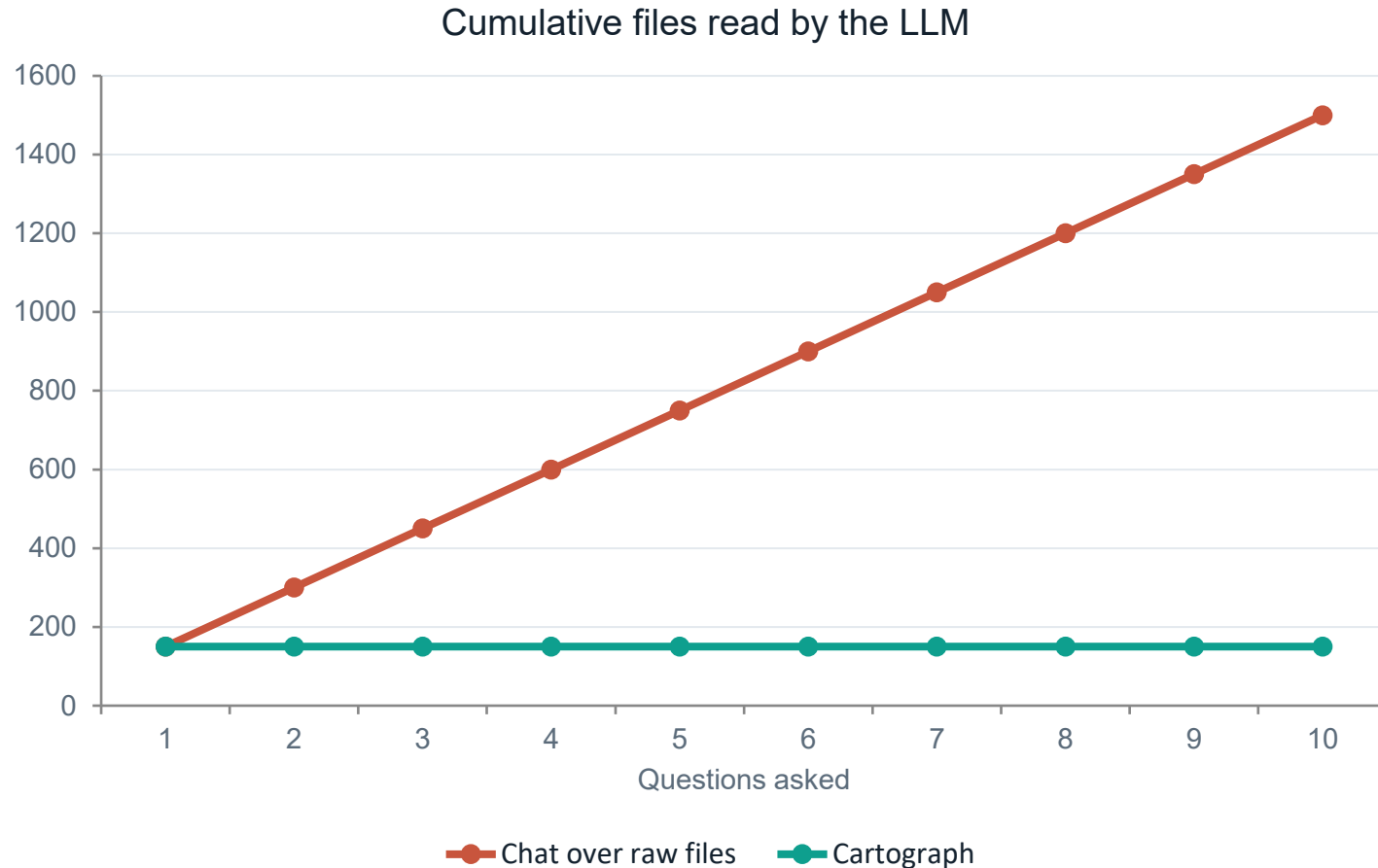
The Planner sees the whole tree first, so a capped run analyses the entry points rather than the alphabetically first files.

4

Linked, not listed

The Linker joins per-file fragments into one connected graph, so questions can cross file boundaries.

Pay for the crawl once, not per question



Illustrative model: a 150-file repository (the default ANALYSER_MAX_FILES), where the chat approach re-reads it for each question.

1 crawl

per repository, then a small subgraph per question

Durable & shared

The graph outlives the session. The next teammate asks questions without rebuilding the map.

From URL to answers in four steps



Point it at code

Paste a public Git URL or upload a .zip in the web UI.

Watch it think

A live trace shows every agent call, the files picked and the tokens spent.

Explore the graph

Browse the overview, symbols and call graph the run produced.

Ask anything

“Where is auth enforced?” The answer cites the nodes it used.

Who it is for

- New joiners on day one
- Reviewers in unfamiliar code
- Engineers inheriting a service
- Leads writing onboarding docs

Bob built it in one Agent-mode session

3,109

lines of hackathon brief turned into a requirements summary

17

tracked items in Bob's phased build plan

50

files written or changed in Agent mode

168k

tokens of context used without losing track



Document understanding

Pulled the deliverables, word limits and evidence rules out of the raw brief.



Plan

Scaffold → models → LLM clients → stores → agents → orchestrator → API/UI → tests → deploy.



Full-repo context

Caught that the store and graph tools no longer matched the orchestrator's StoredNode and write() contract, and rewrote both.



Bob inside the product

LLM_PROVIDER=bobshell routes every agent call through Bob Shell.
.bobignore keeps secrets out of Bob's context.

IBM-first, provider-agnostic, tested offline

IBM stack



IBM Bob 2.0

Planned and wrote the codebase in Agent mode



Bob Shell

Selectable runtime provider: LLM_PROVIDER=bobshell



watsonx.ai

Default provider via the ibm-watsonx-ai SDK, Granite 3 8B Instruct, temperature 0

Switching models is a single environment variable. No agent code changes.

34

tests, all offline: fake LLM, in-memory store, no keys

4

deploy targets: Docker, HF Spaces, Render, Railway

0

secrets in Bob's context, enforced by .bobignore

Cartograph makes reading a codebase a one-time cost.

What's next

- watsonx Orchestrate agent over /analyse and /ask, to ask the graph from chat
- Incremental re-analysis from git diff, so the graph stays current
- PR-review mode: show the parts of the graph a diff touches

github.com/Poornasri2003/code-analyser

